PROGRAMMING IN JAVA

Campus Smart-Parking System

CSA09 | CO1 | Assignment 1

Design, Implement and Validate an Object-Oriented Model

Tasks 1–12 | Total Marks: 50 | Individual

Field	Details
Student name	M. Farah Afrin
Register number	192511434
Submission date	26-10-2026

Contents and Overview

Task	Topic	Marks
1	Problem analysis and objects	4
2	Data representation	3
3	Control logic and pseudocode	4
4	Classes, objects and constructors	4
5	Static members and object life cycle	3
6	Inheritance hierarchy	4
7	Method overriding and overloading	4
8	Polymorphism	4
9	Abstraction and interfaces	4
10	Packages and encapsulation	3
11	Implementation, testing and defence	5
12	Day-4 design challenge	8

Facts used throughout

- Packages: parking.model, parking.service and parking.app.
- Base rates in the current code: TwoWheeler ₹10/hour and Car ₹30/hour.
- ElectricCar inherits the car parking rate and charges ₹12/kWh for charging.
- Hourly billing rounds any started hour upward: hours = (minutes + 59) / 60.
- Task 12 test output uses ElectricBike flat parking fee ₹15 and one-day pass price ₹100. These are design assumptions because the assignment wording does not prescribe the amounts.
- Current slot allocation uses a jagged String[][] array; floor capacities can differ.

Task 1: Understand the Problem and Identify the Objects

Inputs and outputs:

- Inputs: registration number, vehicle type, parking duration, floor capacities, charging energy, monthly-pass status or visitor one-day-pass status, and entry time for time-dependent pricing.
- Outputs: allocated floor and slot, allocation/release status, parking fee, charging fee, pass cost and total payment.

Candidate classes and responsibilities

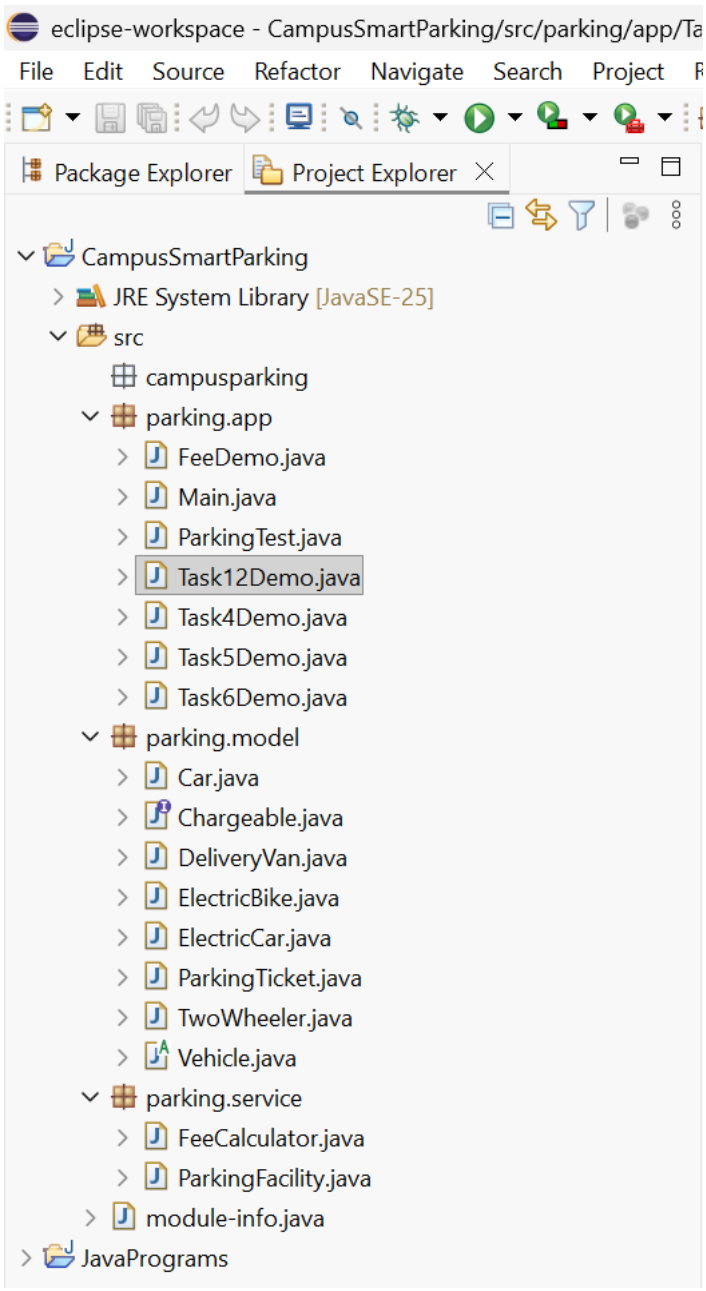
Class / interface	Responsibility
Vehicle (abstract)	Stores validated registration number and calculates hourly parking fees using ratePerHour().
TwoWheeler	Represents a standard two-wheeler at ₹10/hour.
Car	Represents a standard car at ₹30/hour.
ElectricCar	Extends Car and implements Chargeable for charging fees.
ElectricBike	Applies the flat parking fee used in Task 12 tests: ₹15 per visit.
DeliveryVan	Uses double the car rate during 09:00–17:00 and the normal car rate outside that period.
Chargeable	Defines chargingFee(kWh).
ParkingFacility	Allocates and releases registration numbers in a jagged slot array.
FeeCalculator	Combines parking fee, pass exemption and charging fee.
ParkingTicket	Represents parking-session details where used in the project.
Task12Demo / ParkingTest	Runs demonstrations and test scenarios.

Business rules:

- A slot can hold only one registration at a time.
- The same registration cannot be allocated twice while already parked.
- Allocation returns the first available slot, searching floor by floor.
- Invalid floor/slot and releasing an empty slot return false.
- For hourly pricing, 61 minutes is billed as two hours.
- A monthly pass waives parking fees in the current FeeCalculator; charging remains separate.
- Task 12 adds a flat e-bike fee, time-based delivery-van pricing, a one-day visitor pass and five unequal floor capacities.

Assumptions:

The electric-bike parking fee is ₹15 and the visitor one-day pass costs ₹100, matching the Task12Demo output supplied for this report. The pass waives the visitor's parking fee; the pass purchase cost remains ₹100. The delivery-van test uses the time-of-day rule shown by the supplied output. Confirm any unspecified edge cases with the lecturer.


eclipse-workspace - CampusSmartParking/src/parking/app/Ta
File
Edit
Source
Refactor
Navigate
Search
Project
Package Explorer
Project Explorer
CampusSmartParking
JRE System Library [JavaSE-25]
src
campusparking
parking.app
FeeDemo.java
Main.java
ParkingTest.java
Task12Demo.java
Task4Demo.java
Task5Demo.java
Task6Demo.java
parking.model
Car.java
Chargeable.java
DeliveryVan.java
ElectricBike.java
ElectricCar.java
ParkingTicket.java
TwoWheeler.java
Vehicle.java
parking.service
FeeCalculator.java
ParkingFacility.java
module-info.java
JavaPrograms

Task 2: Choose the Data Representation

Field	Type	Reason
Registration number	String	Alphanumeric identifier for a vehicle.
Floor and slot	int	Whole-number array indices.
Floor occupancy	String[][]	Jagged array supports different slot counts on each floor.
Duration	int minutes	Used in integer ceiling calculation for started hours.
Parking / charging fee	double	Matches current program outputs such as ₹30.0 and ₹36.0.
Charging capability	Chargeable interface	Separates the ability to charge from vehicle inheritance.

Why a jagged array?

The facility is represented by one array row per floor, with a separate length for each floor. This directly supports the Task 12 capacities of 10, 15, 12, 20 and 8 slots. A rectangular array would force all floors to have the same length or leave unused cells.

Fee rounding:

Duration	Billable hours	Car fee at ₹30/hour
60 minutes	1	₹30
61 minutes	2	₹60
130 minutes	3	₹90

```
int hours = (minutes + 59) / 60;\nreturn hours * ratePerHour();
```

Complexity:

Let S be the total number of slots. Allocation may scan all slots, so worst-case time is O(S). Slot storage is O(S). Duplicate detection by scanning is also O(S); a registration lookup map could improve average lookup time to O(1).

<terminated> Task12Demo [Java Application] C:\Users\Farah Amin\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_25.0.4.v20260820-0822\jre

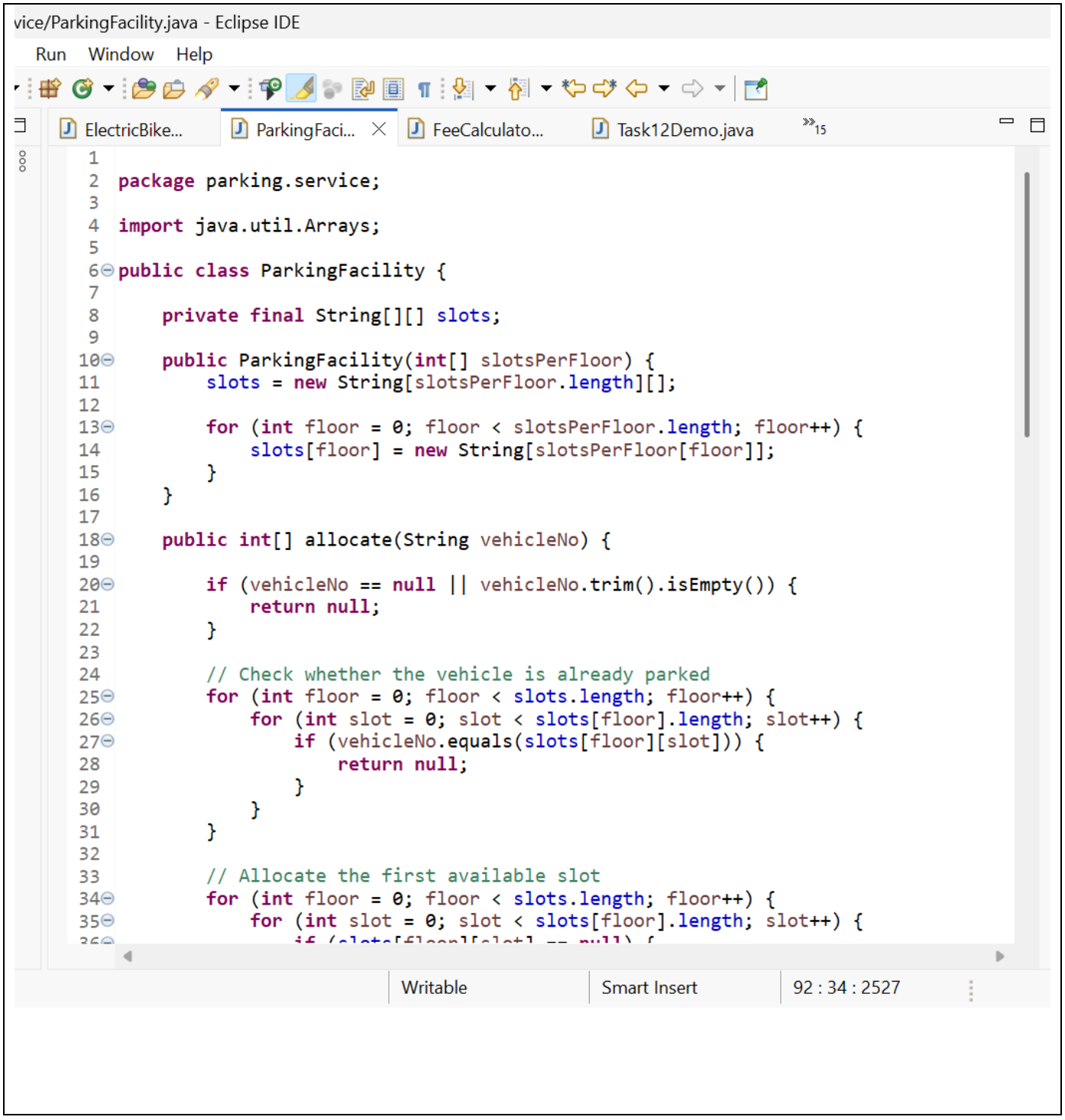
```
TASK 12 - DESIGN CHALLENGE
==========================

1. ELECTRIC BIKE TEST
Parking for 2 hours: Rs. 15.0
Parking for 5 hours: Rs. 15.0

2. DELIVERY VAN TEST
Parking for 2 hours at 10 AM: Rs. 120.0
Parking for 2 hours at 6 PM: Rs. 60.0

3. VISITOR ONE-DAY PASS TEST
One-day pass price: Rs. 100.0
Parking fee with pass: Rs. 0.0
Total paid with pass: Rs. 100.0
Parking fee without pass: Rs. 60.0
Parking fee with monthly pass: Rs. 0.0

4. FIVE-FLOOR PARKING TEST
Floor 0: [null, null, null, null, null, null, null, null, null, null]
Floor 1: [null, null, null, null, null, null, null, null, null, null, null, null, null, null, null]
Floor 2: [null, null, null, null, null, null, null, null, null, null, null, null]
Floor 3: [null, null, null, null, null, null, null, null, null, null, null, null, null, null,
```


vice/ParkingFacility.java - Eclipse IDE
Run Window Help
ElectricBike...
ParkingFaci...
FeeCalculato...
Task12Demo.java
15
1 package parking.service;
2
3 import java.util.Arrays;
4
5 public class ParkingFacility {
6
7 private final String[][] slots;
8
9 public ParkingFacility(int[] slotsPerFloor) {
10 slots = new String[slotsPerFloor.length][];
11
12 for (int floor = 0; floor < slotsPerFloor.length; floor++) {
13 slots[floor] = new String[slotsPerFloor[floor]];
14 }
15 }
16
17 public int[] allocate(String vehicleNo) {
18
19 if (vehicleNo == null || vehicleNo.trim().isEmpty()) {
20 return null;
21 }
22
23 // Check whether the vehicle is already parked
24 for (int floor = 0; floor < slots.length; floor++) {
25 for (int slot = 0; slot < slots[floor].length; slot++) {
26 if (vehicleNo.equals(slots[floor][slot])) {
27 return null;
28 }
29 }
30 }
31
32 // Allocate the first available slot
33 for (int floor = 0; floor < slots.length; floor++) {
34 for (int slot = 0; slot < slots[floor].length; slot++) {
35
Writable
Smart Insert
92 : 34 : 2527

Task 3: Design the Control Logic

Allocation pseudocode — current ParkingFacility logic:

```
ALGORITHM Allocate(vehicleNo)
INPUT: registration number vehicleNo
OUTPUT: [floor, slot] or NULL
IF vehicleNo is null or blank THEN
   RETURN NULL
END IF
FOR each floor f
   FOR each slot s on floor f
      IF vehicleNo already exists in any slot THEN
         RETURN NULL
      END IF
   END FOR
END FOR
FOR each floor f
   FOR each slot s on floor f
      IF slots[f][s] is NULL THEN
         slots[f][s] <- vehicleNo
         RETURN [f, s]
      END IF
   END FOR
END FOR
RETURN NULL   // facility full
END
```

Release pseudocode:

```
ALGORITHM Release(floor, slot)
IF floor is outside valid floor range THEN RETURN false
IF slot is outside the selected floor's range THEN RETURN false
IF slots[floor][slot] is NULL THEN RETURN false
slots[floor][slot] <- NULL
RETURN true
END
```

Fee pseudocode:

```
ALGORITHM HourlyFee(minutes, rate)
IF minutes <= 0 THEN RETURN 0
hours <- (minutes + 59) DIV 60
RETURN hours * rate
END
```

The duplicate check is included because the current ParkingFacility rejects repeated registration numbers. The exact error-handling policy should match the submitted Java method.

```
        // Allocate the first available slot
        for (int floor = 0; floor < slots.length; floor++) {
            for (int slot = 0; slot < slots[floor].length; slot++) {
                if (slots[floor][slot] == null) {
                    slots[floor][slot] = vehicleNo;
                    return new int[]{floor, slot};
                }
            }
        }

        // No available slot
        return null;
    }

    public boolean release(int floor, int slot) {

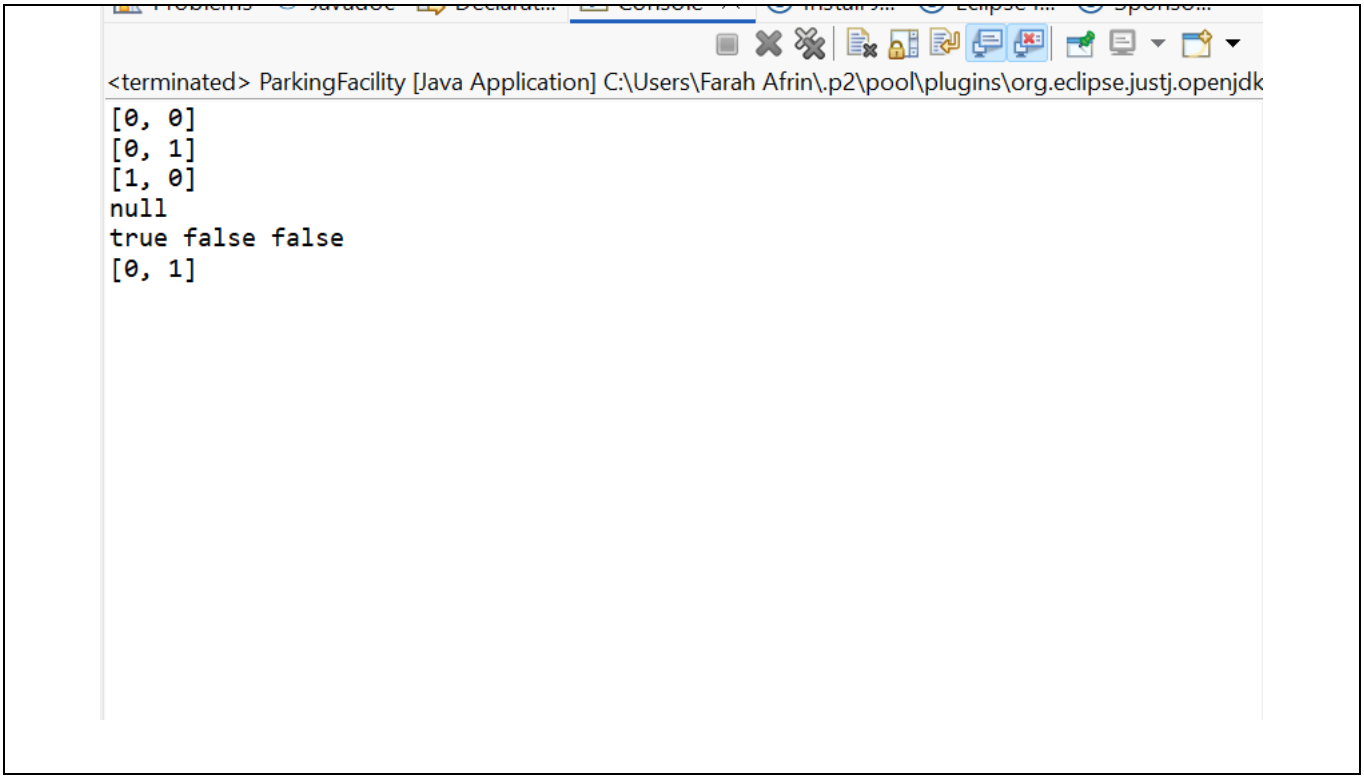
        if (floor < 0 || floor >= slots.length) {
            return false;
        }

        if (slot < 0 || slot >= slots[floor].length) {
            return false;
        }

        if (slots[floor][slot] == null) {
            return false;
        }

        slots[floor][slot] = null;
        return true;
    }

    public void displaySlots() {
```


<terminated> ParkingFacility [Java Application] C:\Users\Farah Afrin\.p2\pool\plugins\org.eclipse.justj.openjdk
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]

Task 4: Design Classes, Objects and Constructors

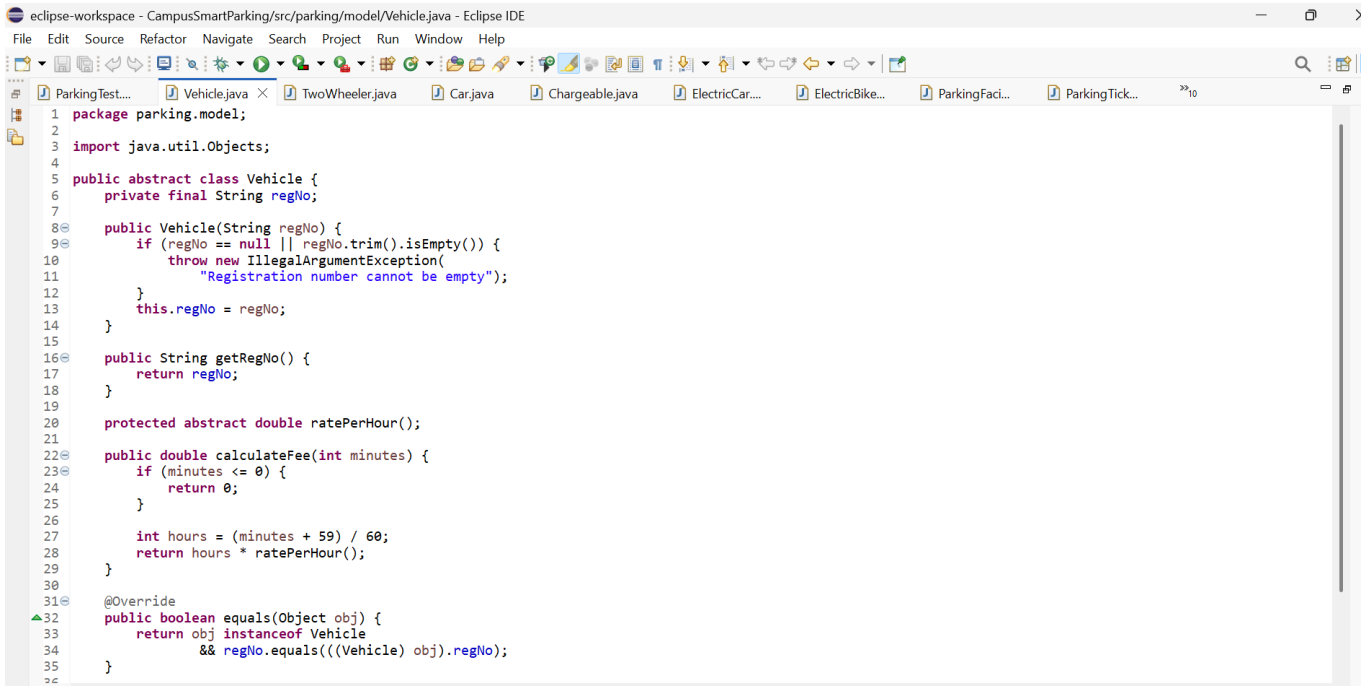
Class	Important state / methods
Vehicle	private final regNo; getRegNo(); calculateFee(minutes); abstract ratePerHour().
TwoWheeler	Constructor calls super(regNo); ratePerHour() returns 10.0.
Car	Constructor calls super(regNo); ratePerHour() returns 30.0.
ElectricCar	Extends Car; chargingFee(kWh) returns kWh × 12.0 for positive kWh.
ElectricBike	Extends TwoWheeler in the current project; Task 12 flat fee overrides hourly calculation.
ParkingFacility	Constructor receives slotsPerFloor and creates a jagged String[][] array.
FeeCalculator	Static method combines parking, pass and charging amounts.

Constructor validation

Vehicle rejects null or blank registration numbers. Subclasses call super(regNo), reusing the shared validation. ParkingFacility creates each floor row using the capacity supplied for that floor. Inputs should be checked before the object is used.

Object lifecycle

A vehicle object is created from a registration number. ParkingFacility stores the registration when allocation succeeds and clears it when release succeeds. The current simple slot model does not itself retain a full ticket history.

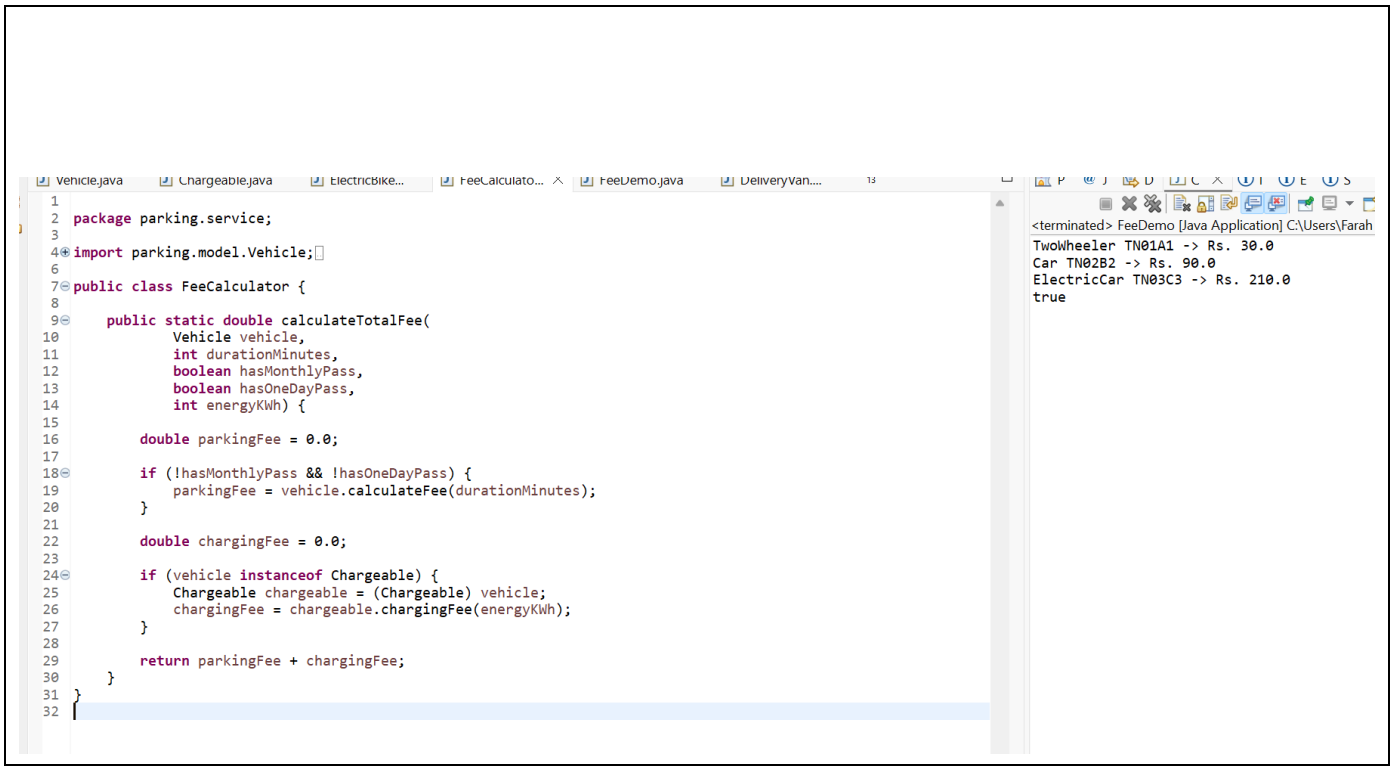

eclipse-workspace - CampusSmartParking/src/parking/model/Vehicle.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
ParkingTest.... Vehicle.java TwoWheeler.java Car.java Chargeable.java ElectricCar.... ElectricBike... ParkingFaci... ParkingTick...
1 package parking.model;
2
3 import java.util.Objects;
4
5 public abstract class Vehicle {
6 private final String regNo;
7
8 public Vehicle(String regNo) {
9 if (regNo == null || regNo.trim().isEmpty()) {
10 throw new IllegalArgumentException(
11 "Registration number cannot be empty");
12 }
13 this.regNo = regNo;
14 }
15
16 public String getRegNo() {
17 return regNo;
18 }
19
20 protected abstract double ratePerHour();
21
22 public double calculateFee(int minutes) {
23 if (minutes <= 0) {
24 return 0;
25 }
26
27 int hours = (minutes + 59) / 60;
28 return hours * ratePerHour();
29 }
30
31 @Override
32 public boolean equals(Object obj) {
33 return obj instanceof Vehicle
34 && regNo.equals(((Vehicle) obj).regNo);
35 }

Task 5: Apply Static Members and Object Life Cycle

FeeCalculator.calculateTotalFee is static in the current code because it performs a calculation from supplied arguments and does not store per-instance state. The ParkingFacility slots array is instance state: each ParkingFacility object has its own occupancy.

Design observations

- Static methods are suitable for stateless utility calculations.
- The current slot array is created in the constructor and becomes eligible for garbage collection when its ParkingFacility object is unreachable.
- A production ticket counter should be persistent and collision-safe; the current simple project does not implement a database-backed ticket sequence.
- Avoid introducing a static counter unless the submitted code actually uses one.



```
Vehicle.java  Chargeable.java  ElectricBike...  FeeCalculato...  FeeDemo.java  DeliveryVan....

1
2  package parking.service;
3
4  import parking.model.Vehicle;
6
7  public class FeeCalculator {
8
9      public static double calculateTotalFee(
10             Vehicle vehicle,
11             int durationMinutes,
12             boolean hasMonthlyPass,
13             boolean hasOneDayPass,
14             int energyKWh) {
15
16         double parkingFee = 0.0;
17
18         if (!hasMonthlyPass && !hasOneDayPass) {
19             parkingFee = vehicle.calculateFee(durationMinutes);
20         }
21
22         double chargingFee = 0.0;
23
24         if (vehicle instanceof Chargeable) {
25             Chargeable chargeable = (Chargeable) vehicle;
26             chargingFee = chargeable.chargingFee(energyKWh);
27         }
28
29         return parkingFee + chargingFee;
30     }
31 }
32
```

```
<terminated> FeeDemo [Java Application] C:\Users\Farah
TwoWheeler TN01A1 -> Rs. 30.0
Car TN02B2 -> Rs. 90.0
ElectricCar TN03C3 -> Rs. 210.0
true
```

Task 6: Design the Inheritance Hierarchy

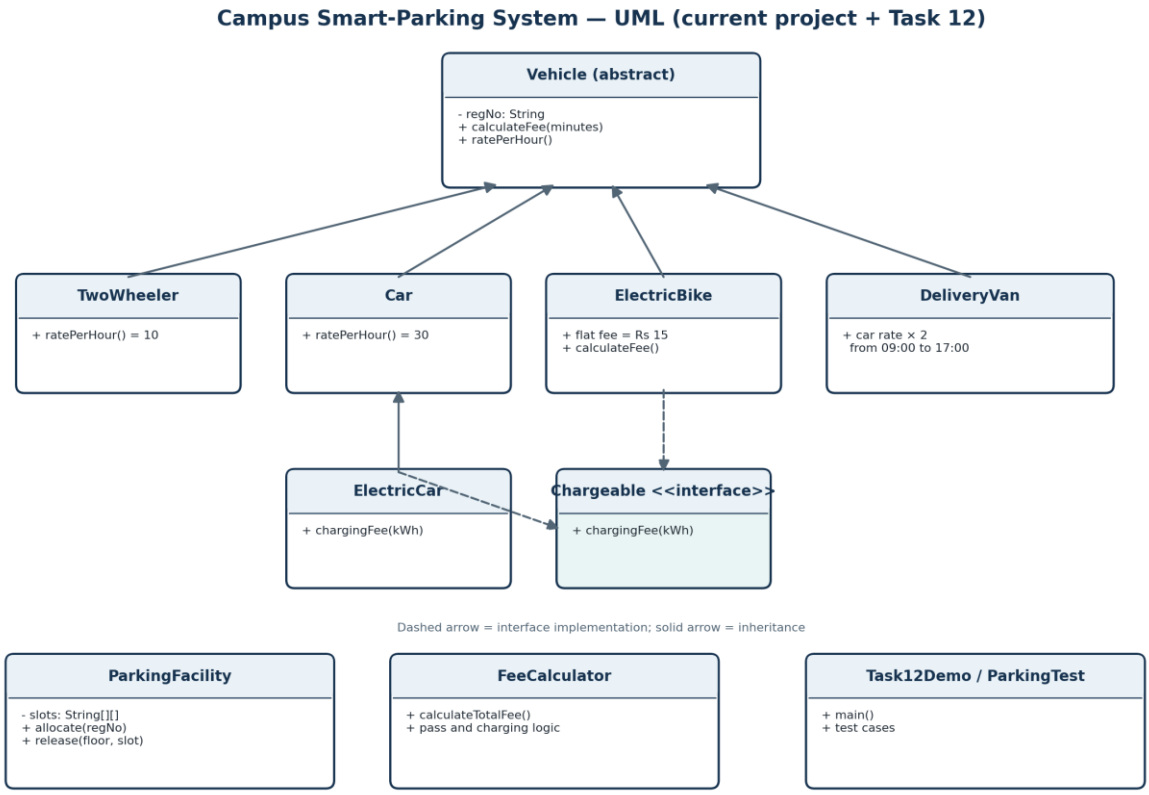
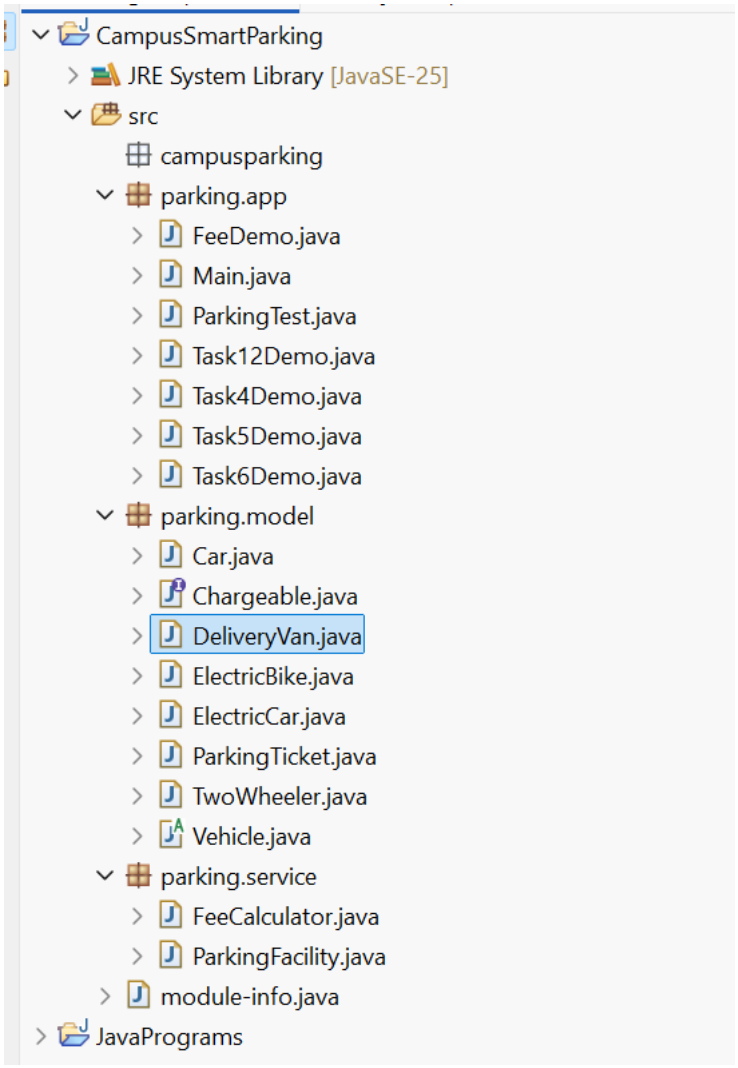


Figure 6 — UML class diagram adapted to the current project and Task 12 vehicle additions.

Explanation

- Vehicle is abstract and stores common registration data.
- TwoWheeler and Car inherit from Vehicle and supply their hourly rates.
- ElectricCar extends Car and implements Chargeable.
- ElectricBike uses a flat fee for parking under Task 12.
- DeliveryVan applies the time-dependent pricing rule.
- ParkingFacility and FeeCalculator are service classes; the application package contains runnable demos.


CampusSmartParking
JRE System Library [JavaSE-25]
src
campusparking
parking.app
FeeDemo.java
Main.java
ParkingTest.java
Task12Demo.java
Task4Demo.java
Task5Demo.java
Task6Demo.java
parking.model
Car.java
Chargeable.java
DeliveryVan.java
ElectricBike.java
ElectricCar.java
ParkingTicket.java
TwoWheeler.java
Vehicle.java
parking.service
FeeCalculator.java
ParkingFacility.java
module-info.java
JavaPrograms

Task 7: Override and Specialise Behaviour

Method overriding

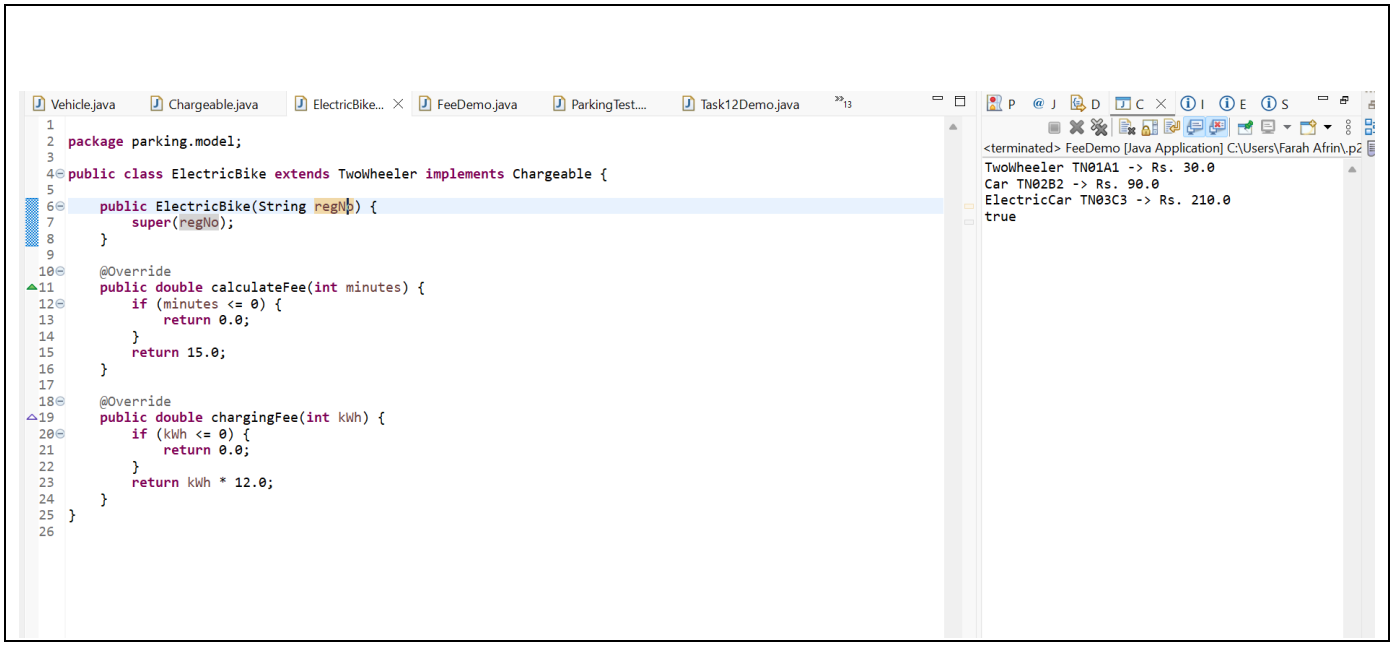
Vehicle.calculateFee(minutes) uses the ratePerHour() method supplied by the actual subclass. ElectricBike overrides the parking-fee calculation for its flat-fee rule. DeliveryVan needs a time-aware calculation to select between the normal car rate and double rate during the specified hours.

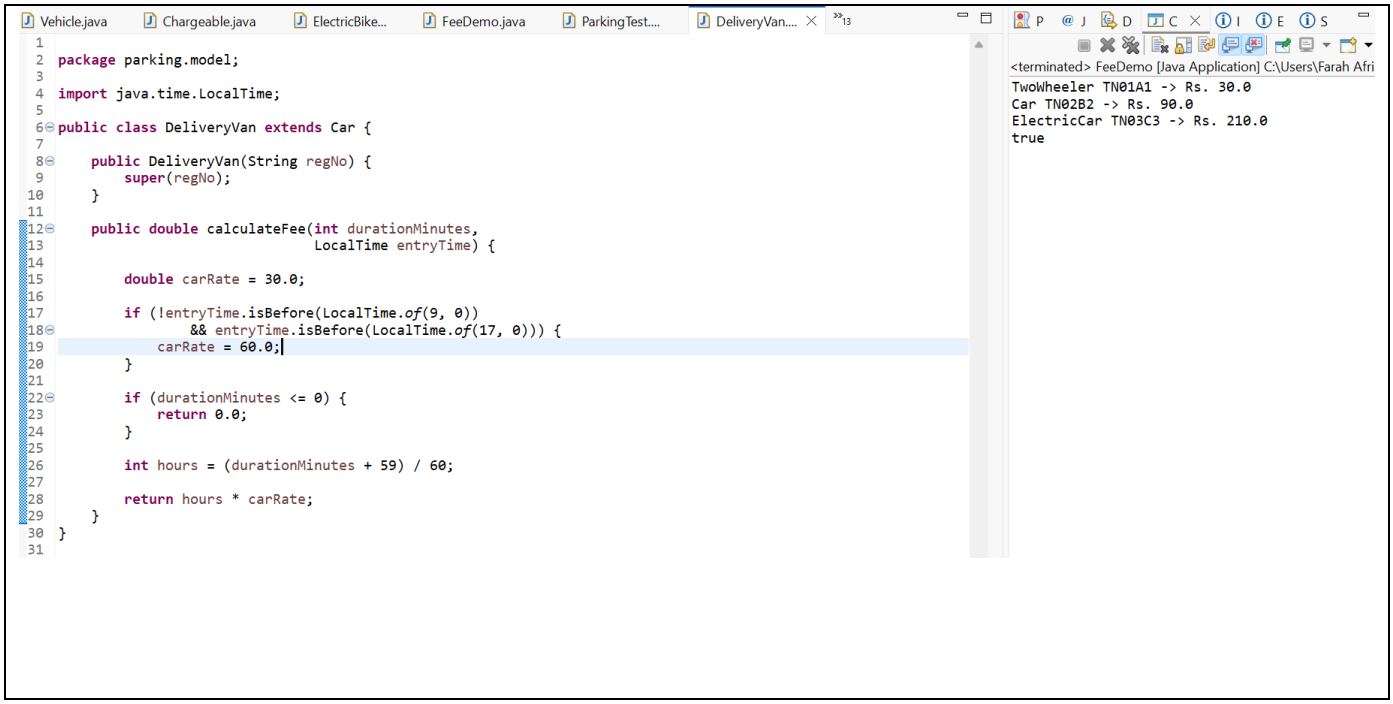
Overloading and overriding

Overloading	Overriding
Same method name with different parameter lists, such as minute-based and time-based fee methods if both are implemented.	A subclass provides the implementation of an inherited method with the same signature.
Resolved by the compiler using argument types.	Runtime dispatch selects the implementation for the actual object.

Equality

Vehicle.equals() compares registration numbers, and hashCode() uses the same identity field. This means two vehicle objects with the same registration number compare equal, consistent with registration-based identity in the current code.






```java

package parking.model;

import java.time.LocalTime;

public class DeliveryVan extends Car {

    public DeliveryVan(String regNo) {
        super(regNo);
    }

    public double calculateFee(int durationMinutes,
                               LocalTime entryTime) {

        double carRate = 30.0;

        if (!entryTime.isBefore(LocalTime.of(9, 0))
                && entryTime.isBefore(LocalTime.of(17, 0))) {
            carRate = 60.0;
        }

        if (durationMinutes <= 0) {
            return 0.0;
        }

        int hours = (durationMinutes + 59) / 60;

        return hours * carRate;
    }
}

```

Task 8: Polymorphism

A Vehicle reference can refer to a TwoWheeler, Car or ElectricCar. Calling an overridden method uses the implementation belonging to the actual object. This keeps the calling code from needing a long chain of vehicle-type checks for hourly parking rates.

Example	Calculation	Expected result
TwoWheeler, 130 minutes	3 × ₹10	₹30.0
Car, 130 minutes	3 × ₹30	₹90.0
ElectricCar, 60 minutes + 10 kWh	₹30 + (10 × ₹12)	₹150.0
ElectricBike, 2 or 5 hours	Flat-fee assumption	₹15.0

Use the actual output from your latest FeeDemo and Task12Demo when completing the evidence section. Do not treat illustrative examples as executed results.

```
1
2  package parking.app;
3
4  import parking.model.Car;
9
10 public class FeeDemo {
11     public static void main(String[] args) {
12
13         Vehicle[] parked = {
14             new TwoWheeler("TN01A1"),
15             new Car("TN02B2"),
16             new ElectricCar("TN03C3")
17         };
18
19         for (Vehicle v : parked) {
20             double fee = v.calculateFee(130);
21
22             if (v instanceof Chargeable) {
23                 fee += ((Chargeable) v).chargingFee(10);
24             }
25
26             System.out.println(
27                 v.getClass().getSimpleName() + " "
28                 + v.getRegNo() + " -> Rs. " + fee
29             );
30         }
31
32         System.out.println(
33             new Car("TN02B2").equals(
34                 new ElectricCar("TN02B2")
35             )
36         );
37     }
38 }
39
```

```
TwoWheeler TN01A1 -> Rs. 30.0
Car TN02B2 -> Rs. 90.0
ElectricCar TN03C3 -> Rs. 210.0
true
```

Task 9: Introduce Abstraction and Interfaces

Abstract class Vehicle

Vehicle contains shared state and the common fee-calculation template. The abstract ratePerHour() method requires concrete hourly vehicle categories to provide their rate.

Chargeable interface

Chargeable represents the capability to calculate charging fees. ElectricCar implements it, and FeeCalculator checks this capability instead of assuming every vehicle can be charged.

Why separate abstraction and capability?

- The abstract class shares state and common behaviour.
- The interface describes a capability that only selected vehicle types need.
- The fee service can add charging costs without duplicating the charging formula for each vehicle class.

Vehicle.java

```
package parking.model;

import java.util.Objects;

public abstract class Vehicle {
   private final String regNo;

   public Vehicle(String regNo) {
      if (regNo == null || regNo.trim().isEmpty()) {
         throw new IllegalArgumentException(
            "Registration number cannot be empty");
      }
      this.regNo = regNo;
   }

   public String getRegNo() {
```

```
        return regNo;
    }

    protected abstract double ratePerHour();

    public double calculateFee(int minutes) {
        if (minutes <= 0) {
            return 0;
        }

        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }

    @Override
    public boolean equals(Object obj) {
        return obj instanceof Vehicle
                && regNo.equals(((Vehicle) obj).regNo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(regNo);
    }
}
```

```java
package parking.model;

import java.util.Objects;

public abstract class Vehicle {
    private final String regNo;

    public Vehicle(String regNo) {
        if (regNo == null || regNo.trim().isEmpty()) {
            throw new IllegalArgumentException(
                "Registration number cannot be empty");
        }
        this.regNo = regNo;
    }

    public String getRegNo() {
        return regNo;
    }

    protected abstract double ratePerHour();

    public double calculateFee(int minutes) {
        if (minutes <= 0) {
            return 0;
        }

        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }

    @Override
    public boolean equals(Object obj) {
        return obj instanceof Vehicle
                && regNo.equals(((Vehicle) obj).regNo);
    }
```

```
TwoWheeler TN01A1 -> Rs. 30.0
Car TN02B2 -> Rs. 90.0
ElectricCar TN03C3 -> Rs. 210.0
true
```

Charageable.java

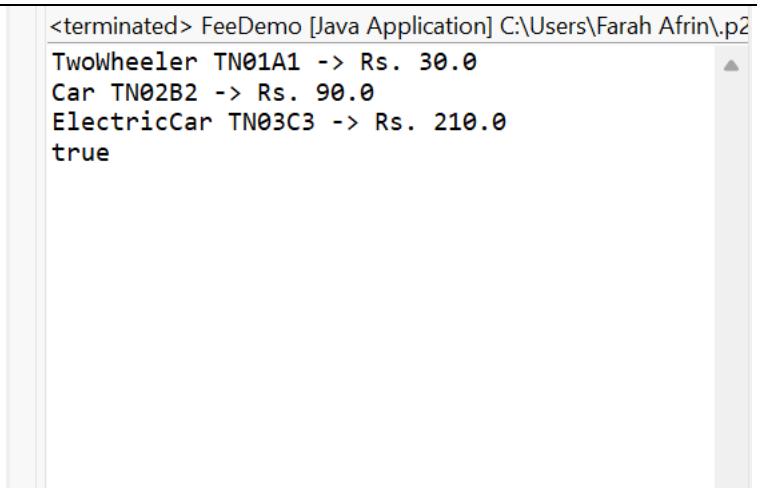
```java
package parking.model;

public interface Chargeable {
    double chargingFee(int kWh);
}
```

Vehicle.java | Chargeable.java | ElectricBike...

```java
package parking.model;

public interface Chargeable {
    double chargingFee(int kWh);
}
```

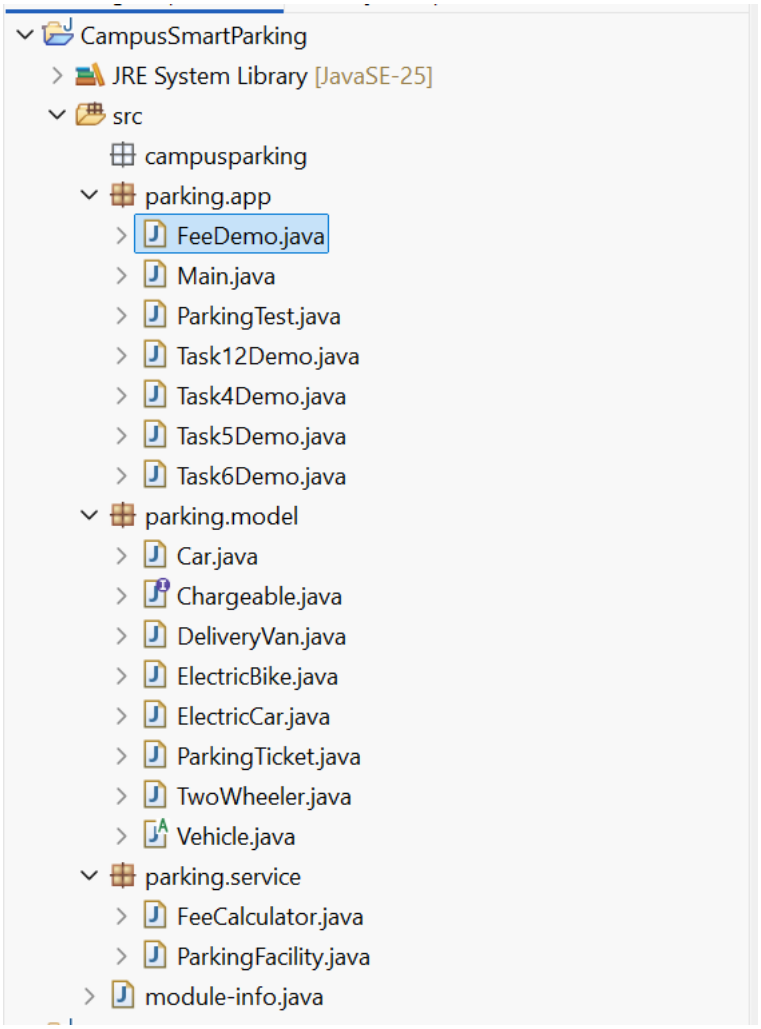

<terminated> FeeDemo [Java Application] C:\Users\Farah Afrin\.p2
TwoWheeler TN01A1 -> Rs. 30.0
Car TN02B2 -> Rs. 90.0
ElectricCar TN03C3 -> Rs. 210.0
true

Task 10: Enforce Encapsulation through Packages and Access Modifiers

Package	Classes in the current project	Purpose
parking.model	Vehicle, TwoWheeler, Car, ElectricCar, ElectricBike, Chargeable, ParkingTicket	Vehicle domain objects and interfaces.
parking.service	ParkingFacility, FeeCalculator	Slot allocation/release and fee calculations.
parking.app	Main, ParkingTest, Task4Demo, Task5Demo, Task6Demo, FeeDemo, Task12Demo	Runnable demonstrations and test cases.

Encapsulation

- Vehicle.regNo is private and final.
- ParkingFacility keeps the slots array private and changes occupancy through allocate() and release().
- FeeCalculator accepts a Vehicle reference and checks Chargeable for charging behaviour.
- Keep package names and class names consistent with the actual Eclipse project.


CampusSmartParking
JRE System Library [JavaSE-25]
src
campusparking
parking.app
FeeDemo.java
Main.java
ParkingTest.java
Task12Demo.java
Task4Demo.java
Task5Demo.java
Task6Demo.java
parking.model
Car.java
Chargeable.java
DeliveryVan.java
ElectricBike.java
ElectricCar.java
ParkingTicket.java
TwoWheeler.java
Vehicle.java
parking.service
FeeCalculator.java
ParkingFacility.java
module-info.java

Task 11: Implement, Test and Defend the Design

Current test evidence available

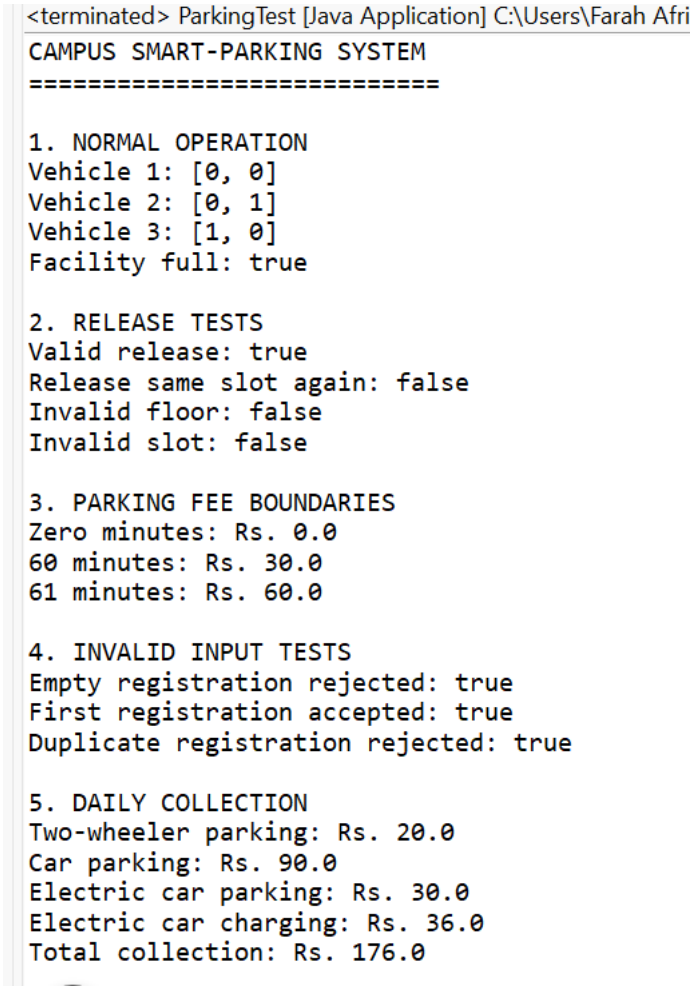
Test	Expected / observed behaviour	Observed output from current project
Allocation	First available slots are assigned	[0, 0], [0, 1], [1, 0]
Full facility	No free slot remains	Allocation returns null
Release	Valid occupied slot released	true
Repeat release	Empty slot cannot be released again	false
Invalid floor/slot	Bounds checked	false
Fee boundaries	Car 0, 60 and 61 minutes	₹0.0, ₹30.0, ₹60.0
Duplicate registration	Same registration cannot be allocated twice	Duplicate rejected
FeeDemo	TwoWheeler, Car, ElectricCar	₹30.0, ₹90.0, ₹210.0 (10 kWh charging)

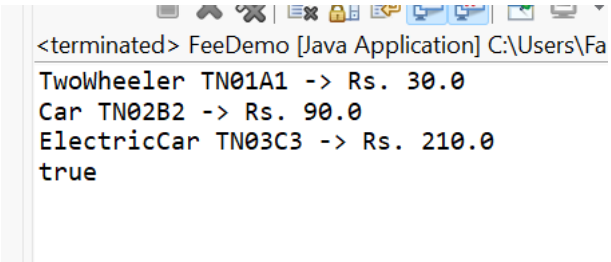
Complexity

Operation	Time	Reason
Allocation	O(S) worst case	May scan every slot.
Release by coordinates	O(1)	Bounds checks and clearing a slot are constant time.
Display slots	O(S)	Every slot is printed.
Storage	O(S)	One cell per parking slot.

Limitations

The current implementation is an in-memory educational prototype. Slot data is lost when the program exits. Financial values use double, which is acceptable for the assignment examples but is not ideal for production money calculations. The van pricing implementation should document how a stay crossing a pricing boundary is treated.


<terminated> ParkingTest [Java Application] C:\Users\Farah Afri
CAMPUS SMART-PARKING SYSTEM
==========
1. NORMAL OPERATION
Vehicle 1: [0, 0]
Vehicle 2: [0, 1]
Vehicle 3: [1, 0]
Facility full: true
2. RELEASE TESTS
Valid release: true
Release same slot again: false
Invalid floor: false
Invalid slot: false
3. PARKING FEE BOUNDARIES
Zero minutes: Rs. 0.0
60 minutes: Rs. 30.0
61 minutes: Rs. 60.0
4. INVALID INPUT TESTS
Empty registration rejected: true
First registration accepted: true
Duplicate registration rejected: true
5. DAILY COLLECTION
Two-wheeler parking: Rs. 20.0
Car parking: Rs. 90.0
Electric car parking: Rs. 30.0
Electric car charging: Rs. 36.0
Total collection: Rs. 176.0


<terminated> FeeDemo [Java Application] C:\Users\Fa
TwoWheeler TN01A1 -> Rs. 30.0
Car TN02B2 -> Rs. 90.0
ElectricCar TN03C3 -> Rs. 210.0
true

Task 12: Re-evaluate and Adapt the Design (Day-4 Design Challenge)

Changed requirements: add electric bikes with a flat parking fee; delivery vans charged at double the car rate during 09:00–17:00; visitor one-day passes; and five floors with different slot counts.

A. Impact analysis (2 marks)

Changed requirement	Effect on the current design
Electric-bike flat fee	ElectricBike requires a flat-fee override rather than the inherited ₹10/hour calculation.
Delivery-van time rule	DeliveryVan needs time-of-day information to apply double the car rate between 09:00 and 17:00.
Visitor one-day pass	FeeCalculator must distinguish a visitor day pass from the existing monthly-pass Boolean.
Five unequal floors	ParkingFacility's jagged String[][] array supports different row lengths; constructor receives a capacity for each floor.

B. Evaluation of the original design (2 marks)

- The original Vehicle hierarchy can be extended with new vehicle classes.
- Pricing is not fully centralised: hourly rates live in vehicle subclasses, while pass and charging logic are in FeeCalculator.
- The slot allocation core can remain unchanged because it does not need to know vehicle pricing categories.
- The current Boolean pass design is less expressive than a Pass interface or dedicated pass class.

C. Adaptation or redesign (2 marks)

Decision: modify the current design rather than rewrite it. Retain Vehicle, Chargeable, ParkingFacility and the package structure. Add the flat-fee behaviour to ElectricBike, add DeliveryVan time-based pricing, and extend FeeCalculator or a pass abstraction to represent the one-day visitor pass. Use the jagged array for five unequal floors.

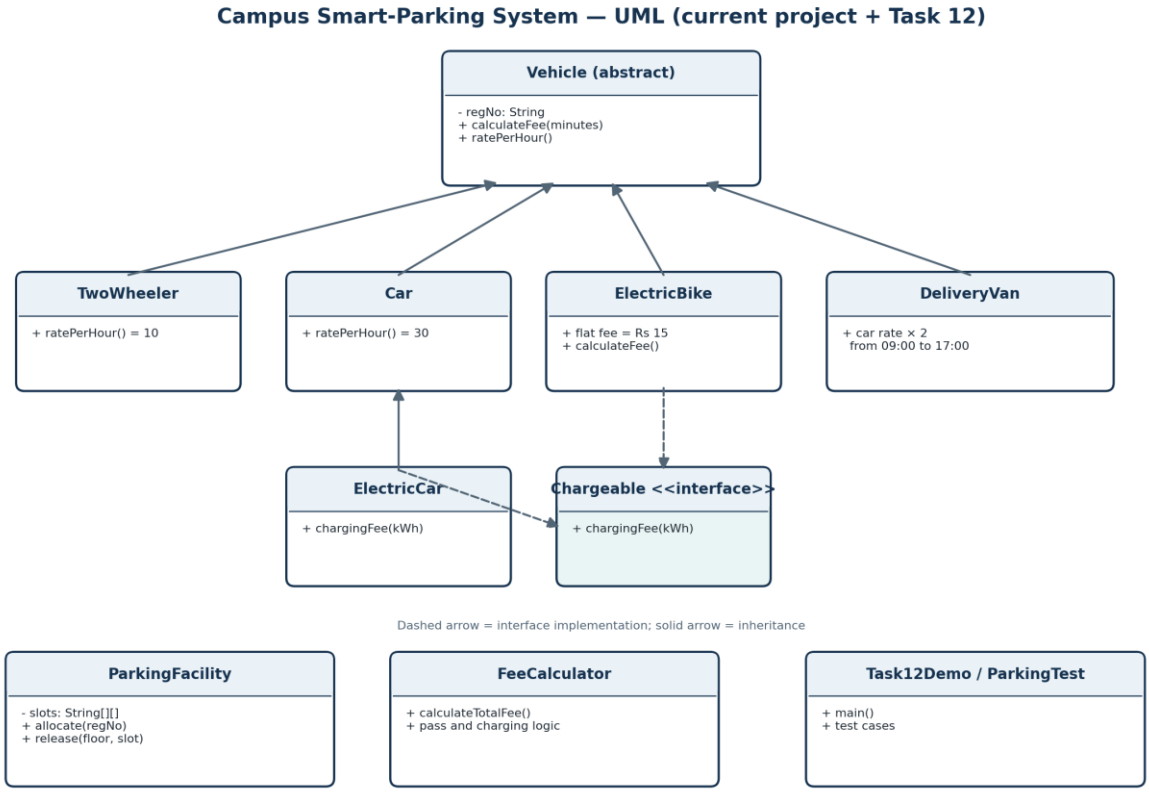


Figure 14 — Revised UML class structure matching the current project and Task 12 additions.

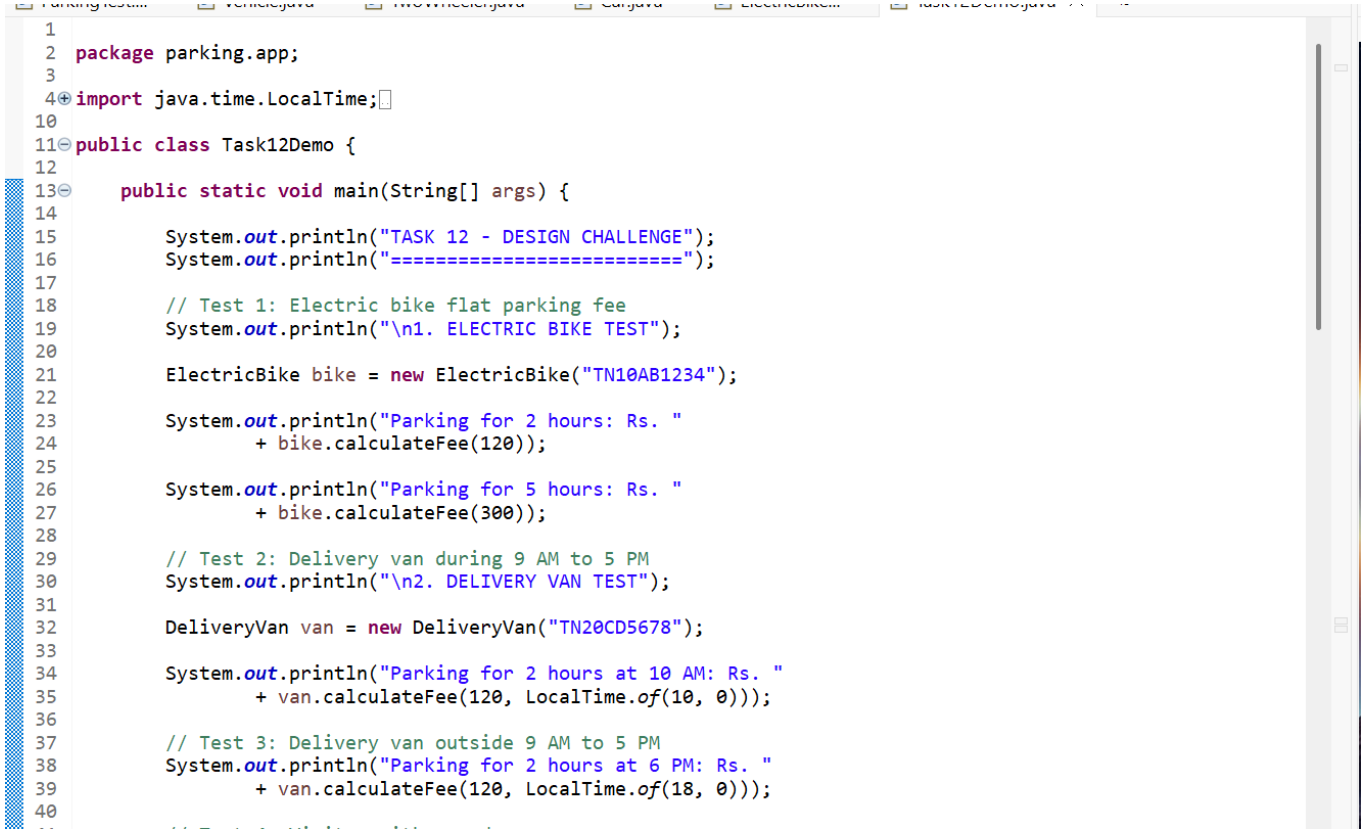
D. Evidence-based defence (2 marks)

Task12Demo test	Output supplied by user
Electric bike parking for 2 hours	₹15.0
Electric bike parking for 5 hours	₹15.0
Delivery van, 2 hours at 10 AM	₹120.0
Delivery van, 2 hours at 6 PM	₹60.0
One-day pass price	₹100.0
Parking fee with one-day pass	₹0.0
Total paid with pass	₹100.0
Visitor without a pass, 2 hours	₹60.0
Monthly-pass parking fee	₹0.0
Five floor capacities	10, 15, 12, 20, 8 slots
Allocation and release	Allocated floor 0, slot 0; release true

Trade-offs and conclusion:

- Flat fees and time-based pricing are easy to add through specialised vehicle classes.

- The jagged array directly represents unequal floor sizes.
- The current pass representation needs improvement if several pass types or expiry rules are added.
- A separate PricingRule strategy could improve reuse later, but adding it now may be unnecessary for this small assignment.



```
package parking.app;

import java.time.LocalTime;

public class Task12Demo {

    public static void main(String[] args) {

        System.out.println("TASK 12 - DESIGN CHALLENGE");
        System.out.println("==========================");

        // Test 1: Electric bike flat parking fee
        System.out.println("\n1. ELECTRIC BIKE TEST");

        ElectricBike bike = new ElectricBike("TN10AB1234");

        System.out.println("Parking for 2 hours: Rs. "
                + bike.calculateFee(120));

        System.out.println("Parking for 5 hours: Rs. "
                + bike.calculateFee(300));

        // Test 2: Delivery van during 9 AM to 5 PM
        System.out.println("\n2. DELIVERY VAN TEST");

        DeliveryVan van = new DeliveryVan("TN20CD5678");

        System.out.println("Parking for 2 hours at 10 AM: Rs. "
                + van.calculateFee(120, LocalTime.of(10, 0)));

        // Test 3: Delivery van outside 9 AM to 5 PM
        System.out.println("Parking for 2 hours at 6 PM: Rs. "
                + van.calculateFee(120, LocalTime.of(18, 0)));

```

package parking.app;

import java.time.LocalTime;
import parking.model.Car;
import parking.model.DeliveryVan;
import parking.model.ElectricBike;
import parking.service.FeeCalculator;
import parking.service.ParkingFacility;

```
public class Task12Demo {

    public static void main(String[] args) {

        System.out.println("TASK 12 - DESIGN CHALLENGE");
        System.out.println("==========================");

        // Test 1: Electric bike flat parking fee
        System.out.println("\n1. ELECTRIC BIKE TEST");

        ElectricBike bike = new ElectricBike("TN10AB1234");

        System.out.println("Parking for 2 hours: Rs. "
                + bike.calculateFee(120));

        System.out.println("Parking for 5 hours: Rs. "
                + bike.calculateFee(300));

        // Test 2: Delivery van during 9 AM to 5 PM
        System.out.println("\n2. DELIVERY VAN TEST");

        DeliveryVan van = new DeliveryVan("TN20CD5678");

        System.out.println("Parking for 2 hours at 10 AM: Rs. "
                + van.calculateFee(120, LocalTime.of(10, 0)));

        // Test 3: Delivery van outside 9 AM to 5 PM
        System.out.println("Parking for 2 hours at 6 PM: Rs. "
                + van.calculateFee(120, LocalTime.of(18, 0)));

        // Test 4: Visitor with one-day pass
        System.out.println("\n3. VISITOR ONE-DAY PASS TEST");

        Car car = new Car("TN30EF9012");
```

```
double dailyPassPrice = 100.0;

double passHolderFee = FeeCalculator.calculateTotalFee(
        car, 120, false, true, 0);

System.out.println("One-day pass price: Rs. "
        + dailyPassPrice);

System.out.println("Parking fee with pass: Rs. "
        + passHolderFee);

System.out.println("Total paid with pass: Rs. "
        + (dailyPassPrice + passHolderFee));

// Test 5: Visitor without a pass
double normalFee = FeeCalculator.calculateTotalFee(
        car, 120, false, false, 0);

System.out.println("Parking fee without pass: Rs. "
        + normalFee);

// Test 6: Monthly pass
double monthlyPassFee = FeeCalculator.calculateTotalFee(
        car, 120, true, false, 0);

System.out.println("Parking fee with monthly pass: Rs. "
        + monthlyPassFee);

// Test 7: Five floors with different slot counts
System.out.println("\n4. FIVE-FLOOR PARKING TEST");

ParkingFacility facility =
        new ParkingFacility(new int[]{10, 15, 12, 20, 8});
```

```
        facility.displaySlots();

        // Test 8: Allocate and release a parking slot
        System.out.println("\n5. ALLOCATION AND RELEASE TEST");

        int[] slot = facility.allocate("TN40GH3456");

        if (slot != null) {
            System.out.println("Allocated floor: " + slot[0]);
            System.out.println("Allocated slot: " + slot[1]);

            System.out.println("Release successful: "
                    + facility.release(slot[0], slot[1]));
        } else {
            System.out.println("Parking facility is full.");
        }

        System.out.println("\nTASK 12 TESTING COMPLETED");
    }
}
```

<terminated> Task12Demo [Java Application] C:\Users\Farah Al

```
TASK 12 - DESIGN CHALLENGE
==========================

1. ELECTRIC BIKE TEST
Parking for 2 hours: Rs. 15.0
Parking for 5 hours: Rs. 15.0

2. DELIVERY VAN TEST
Parking for 2 hours at 10 AM: Rs. 120.0
Parking for 2 hours at 6 PM: Rs. 60.0

3. VISITOR ONE-DAY PASS TEST
One-day pass price: Rs. 100.0
Parking fee with pass: Rs. 0.0
Total paid with pass: Rs. 100.0
Parking fee without pass: Rs. 60.0
Parking fee with monthly pass: Rs. 0.0

4. FIVE-FLOOR PARKING TEST
Floor 0: [null, null, null, null, null, null,
Floor 1: [null, null, null, null, null, null,
Floor 2: [null, null, null, null, null, null,
Floor 3: [null, null, null, null, null, null,
Floor 4: [null, null, null, null, null, null,

5. ALLOCATION AND RELEASE TEST
Allocated floor: 0
Allocated slot: 0
Release successful: true

TASK 12 TESTING COMPLETED
```

<terminated> Task12Demo [Java Application] C:\Users\Farah Al

```
TASK 12 - DESIGN CHALLENGE
==========================

1. ELECTRIC BIKE TEST
Parking for 2 hours: Rs. 15.0
Parking for 5 hours: Rs. 15.0

2. DELIVERY VAN TEST
Parking for 2 hours at 10 AM: Rs. 120.0
Parking for 2 hours at 6 PM: Rs. 60.0

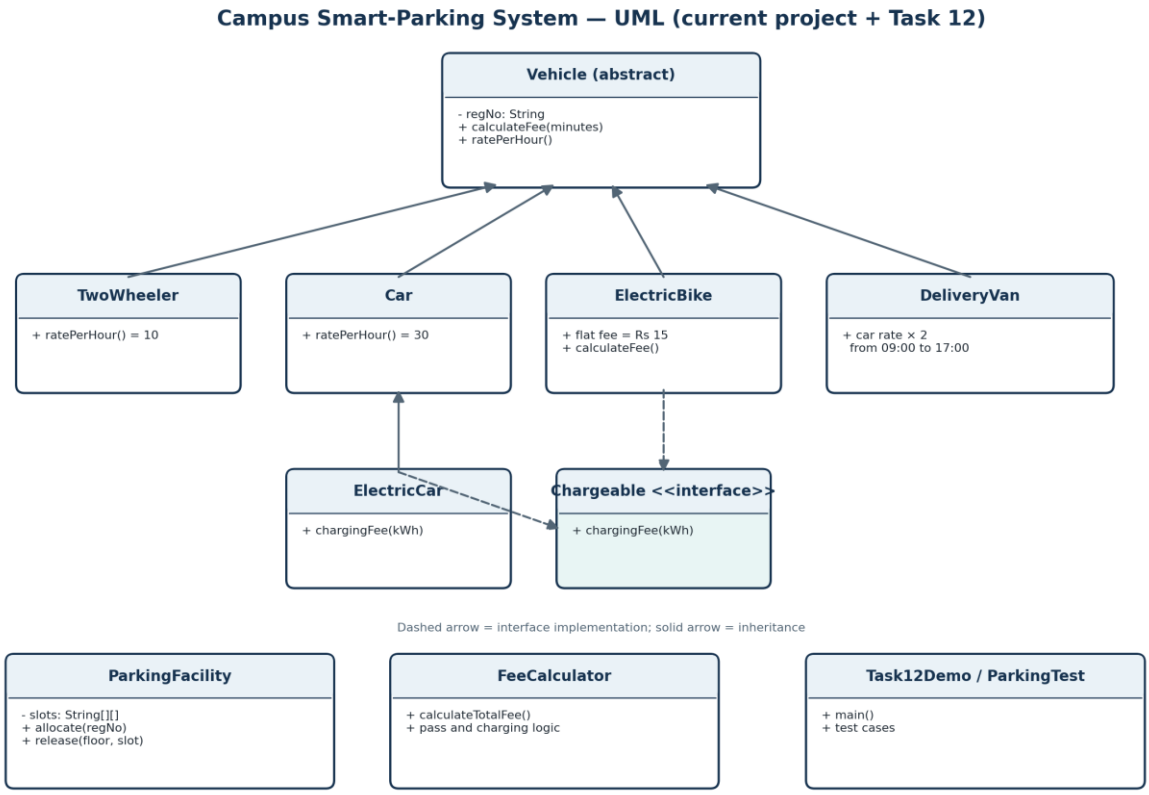
3. VISITOR ONE-DAY PASS TEST
One-day pass price: Rs. 100.0
Parking fee with pass: Rs. 0.0
Total paid with pass: Rs. 100.0
Parking fee without pass: Rs. 60.0
Parking fee with monthly pass: Rs. 0.0

4. FIVE-FLOOR PARKING TEST
Floor 0: [null, null, null, null, null, null,
Floor 1: [null, null, null, null, null, null,
Floor 2: [null, null, null, null, null, null,
Floor 3: [null, null, null, null, null, null,
Floor 4: [null, null, null, null, null, null,

5. ALLOCATION AND RELEASE TEST
Allocated floor: 0
Allocated slot: 0
Release successful: true

TASK 12 TESTING COMPLETED
```

Appendix A: UML Class Diagram



Notation: solid arrows indicate inheritance; dashed arrows indicate interface realization. Verify that every class and method shown is present in the submitted source.